

EXPERIMENT – 2 GO FURTHER – 2

Objective

Digital Password Lock System using Arduino UNO, OLED Display, Keypad, LEDs, and Buzzer

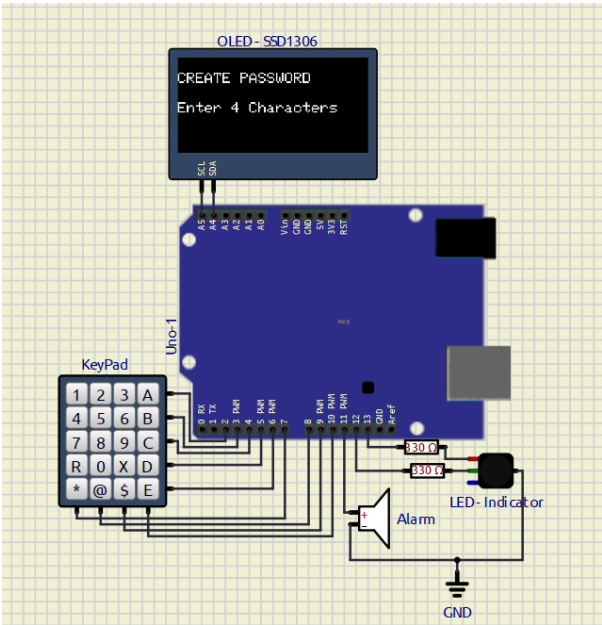
Problem Statement

To design and implement a Digital Password Lock System using Arduino UNO that allows a user to create a 4-character password, authenticate access using the password, provide visual and audio feedback for successful and failed login attempts, and support secure password reset functionality.

Components Required

Sl. No.	Component	Quantity
1	Arduino UNO	1
2	SSD1306 OLED Display (I2C)	1
3	4×5 Matrix Keypad	1
4	Red LED	1
5	Green/Blue LED	1
6	Buzzer	1
7	Breadboard	1
8	Connecting Wires	As Required
9	USB Cable	1

Circuit Diagram



Connections

OLED Display

OLED Pin	Arduino UNO
VCC	5V
GND	GND
SDA	A4
SCL	A5

Keypad Connections

Keypad Pin	Arduino Pin
R1	D2
R2	D3
R3	D4
R4	D5
R5	D6
C1	D7
C2	D8
C3	D9
C4	D10

LED and Buzzer Connections

Component	Arduino Pin
Buzzer	D11
Green LED	D12
Red LED	D13

```
## Code
#include <Wire.h> // I2C communication library
#include <Adafruit_GFX.h> // Graphics library for OLED
#include <Adafruit_SSD1306.h> // SSD1306 OLED library
#include <Keypad.h> // Matrix keypad library

// ===== OLED CONFIGURATION =====
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

// Create OLED display object
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// ===== OUTPUT PINS =====
#define BUZZER 11
#define BLUE_LED 12
#define RED_LED 13

// ===== KEYPAD CONFIGURATION =====
const byte ROWS = 5;
const byte COLS = 4;

// Key mapping for 5x4 keypad
char keys[ROWS][COLS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'R', '0', 'X', 'D'}, // R used for password reset
  {'*', '@', '$', 'E'}
};

// Arduino pins connected to keypad rows
byte rowPins[ROWS] = {2, 3, 4, 5, 6};

// Arduino pins connected to keypad columns
byte colPins[COLS] = {7, 8, 9, 10};

// Create keypad object
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

// ===== GLOBAL VARIABLES =====
String adminPassword = ""; // Stores current password
int attemptsLeft = 3; // Number of login attempts
```

```
// =====  
// Function: Display two lines of text on OLED  
// =====  
void showMessage(String line1, String line2 = "") {  
  
    display.clearDisplay();  
  
    display.setTextSize(1);  
    display.setTextColor(WHITE);  
  
    display.setCursor(0, 10);  
    display.println(line1);  
  
    display.setCursor(0, 30);  
    display.println(line2);  
  
    display.display();  
}  
  
// =====  
// Function: Check if password is valid  
// Only 4-character passwords are accepted  
// =====  
bool isValidPassword(String pass) {  
  
    if (pass.length() != 4)  
        return false;  
  
    return true;  
}  
  
// =====  
// Function: Read 4-character password from keypad  
// Displays '*' instead of actual characters  
// =====  
String readPassword() {  
  
    String pass = "";  
  
    while (pass.length() < 4) {  
  
        char key = keypad.getKey();  
  
        if (key) {  
  
            // Key press beep  
            tone(BUZZER, 1500, 80);  
  
            // Store entered character  
            pass += key;  
  
            // Update OLED display  
            display.clearDisplay();  
        }  
    }  
}
```

```
        display.setTextSize(1);
        display.setTextColor(WHITE);

        display.setCursor(0, 10);
        display.print("ENTER PASSWORD");

        display.setCursor(0, 30);

        // Show stars instead of password
        for (int i = 0; i < pass.length(); i++) {
            display.print("*");
        }

        display.display();
    }
}

delay(500);

return pass;
}

// =====
// Function: Access Granted indication
// Blue LED ON + success beep
// =====
void accessGranted() {

    digitalWrite(BLUE_LED, HIGH);

    tone(BUZZER, 2000, 300);

    showMessage("ACCESS GRANTED");

    delay(3000);

    digitalWrite(BLUE_LED, LOW);
}

// =====
// Function: Access Denied indication
// Red LED ON + error beep
// =====
void accessDenied() {

    digitalWrite(RED_LED, HIGH);

    tone(BUZZER, 500, 500);

    showMessage(
        "WRONG PASSWORD",
        "Attempts Left: " + String(attemptsLeft)
    );
}
```

```
    delay(2000);

    digitalWrite(RED_LED, LOW);
}

// =====
// Function: Alarm Mode
// Activated after 3 wrong attempts
// Infinite loop with buzzer and LED blinking
// =====
void alarmMode() {

    showMessage("SYSTEM LOCKED", "ALARM ACTIVE");

    while (1) {

        digitalWrite(RED_LED, HIGH);
        tone(BUZZER, 700);

        delay(300);

        digitalWrite(RED_LED, LOW);
        noTone(BUZZER);

        delay(300);
    }
}

// =====
// Function: Create initial password
// Runs once during startup
// =====
void createPassword() {

    while (1) {

        showMessage(
            "CREATE PASSWORD",
            "Enter 4 Characters"
        );

        String newPass = readPassword();

        if (isValidPassword(newPass)) {

            adminPassword = newPass;

            tone(BUZZER, 2000, 300);

            showMessage("PASSWORD SAVED");

            delay(2000);
        }
    }
}
```

```
        break;
    }

    else {

        tone(BUZZER, 400, 1000);

        showMessage("INVALID PASSWORD");

        delay(2000);
    }
}

// =====
// Function: Reset password
// User must enter old password first
// =====
void resetPassword() {

    showMessage("RESET MODE", "Enter Old Pass");

    String oldPass = readPassword();

    // Old password correct
    if (oldPass == adminPassword) {

        tone(BUZZER, 2000, 300);

        showMessage("CORRECT", "Set New Password");

        delay(2000);

        while (1) {

            String newPass = readPassword();

            if (isValidPassword(newPass)) {

                adminPassword = newPass;

                tone(BUZZER, 2000, 500);

                showMessage("PASSWORD RESET");

                delay(2000);

                return;
            }

            else {

                tone(BUZZER, 400, 1000);
```

```
        showMessage("INVALID PASSWORD");

        delay(2000);
    }
}

// Old password incorrect
else {

    tone(BUZZER, 300, 1000);

    showMessage("WRONG OLD PASS");

    delay(2000);
}
}

// =====
// SETUP FUNCTION
// Runs once after power ON
// =====
void setup() {

    // Configure output pins
    pinMode(BUZZER, OUTPUT);

    pinMode(BLUE_LED, OUTPUT);
    pinMode(RED_LED, OUTPUT);

    // Initialize I2C
    Wire.begin();

    // Initialize OLED
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);

    display.clearDisplay();
    display.display();

    // Ask user to create password
    createPassword();
}

// =====
// MAIN LOOP
// Password verification system
// =====
void loop() {

    showMessage(
        "ENTER PASSWORD",
        "Attempts Left: " + String(attemptsLeft)
    );
}
```



```
String enteredPassword = "";

// Read 4-character password
while (enteredPassword.length() < 4) {

    char key = keypad.getKey();

    if (key) {

        // Enter password reset mode
        if (key == 'R') {

            resetPassword();

            return;
        }

        tone(BUZZER, 1500, 80);

        enteredPassword += key;

        display.clearDisplay();

        display.setTextSize(1);
        display.setTextColor(WHITE);

        display.setCursor(0, 10);
        display.print("ENTER PASSWORD");

        display.setCursor(0, 30);

        // Display stars
        for (int i = 0; i < enteredPassword.length(); i++) {
            display.print("*");
        }

        display.display();
    }
}

// ===== PASSWORD CHECK =====

if (enteredPassword == adminPassword) {

    // Reset attempts after successful login
    attemptsLeft = 3;

    accessGranted();
}

else {

    // Decrease attempts
    attemptsLeft--;
```

```
    accessDenied();

    // Lock system after 3 failures
    if (attemptsLeft <= 0) {

        alarmMode();
    }
}
}
```

Output

System Startup

OLED Display:

```
SET ADMIN PASS
Enter 4 Characters
```

Password Saved

OLED Display:

```
PASSWORD SAVED
System Ready
```

User Login

OLED Display:

```
ENTER PASSWORD
Unlock The System
```

Correct Password

OLED Display:

```
ACCESS GRANTED
```

- Green LED turns ON.

- Buzzer beeps.

Wrong Password

OLED Display:

```
ACCESS DENIED  
LEFT: 2
```

- Red LED turns ON.
- Remaining attempts decrease.

Alarm Mode

OLED Display:

```
SYSTEM LOCKED  
PRESS R RESET
```

- Red LED blinks continuously.
- Buzzer sounds continuously.

Password Reset

OLED Display:

```
RESET PASSWORD  
Enter Old Pass
```

After successful verification:

```
CORRECT PASSWORD  
Set New Pass
```

Problems Faced

1. OLED display address mismatch (0x3C / 0x3D).
2. Incorrect SDA and SCL wiring.
3. Keypad row and column mapping errors.
4. Undefined LED pin names during coding.
5. OLED text overflow due to long messages.
6. Alarm mode creating an infinite loop.

7. Debugging password reset functionality.
 8. Ensuring proper synchronization between keypad input and OLED display updates.
-

Conclusion

The Digital Password Lock System was successfully implemented using Arduino UNO. The system allows secure password-based access control using a keypad and OLED display. Visual and audio feedback improve user interaction, while the three-attempt security mechanism and alarm mode enhance system security. The password reset feature further increases usability and flexibility. This experiment demonstrates practical applications of embedded systems in electronic security and access control.

Resources

11. Web Resources / References

1. SSD1306 OLED Display Datasheet <https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>
2. Matrix Keypad Working Principle <https://www.electronicsforu.com/technology-trends/learn-electronics/keypad-interfacing-arduino>
3. I2C Communication Protocol Overview <https://lastminuteengineers.com/i2c-lcd-arduino-tutorial/>
4. Password-Based Access Control Systems <https://circuitdigest.com/microcontroller-projects/password-based-door-lock-system-using-arduino>
5. Matrix Keypad Scanning Technique <https://www.geeksforgeeks.org/keypad-interfacing-with-arduino/>
6. Buzzer and Alarm Circuit Concepts <https://components101.com/modules/buzzer-module>